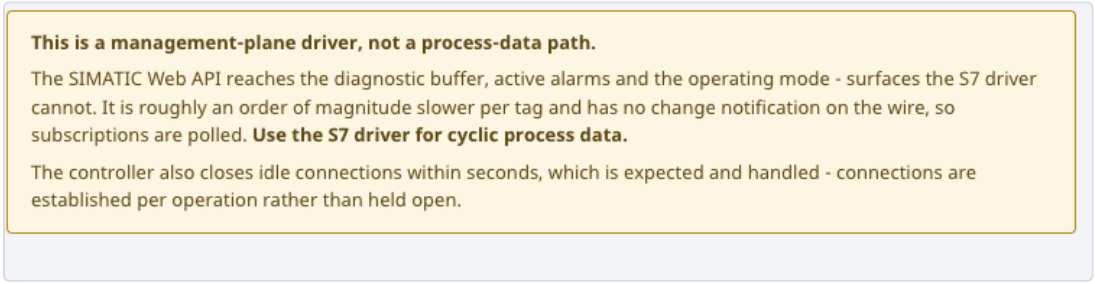
ToddySoft Connect S7 Web API Driver - Ignition 8.3

Ignition 8.3 module for reaching Siemens S7-1200 and S7-1500 controllers over the **SIMATIC Web API** — the controller's own HTTPS/JSON-RPC interface.

> **This is a management-plane driver, not a process-data path.**
>
> The Web API reaches the diagnostic buffer, active alarms and the operating mode — surfaces the S7 driver cannot. In exchange it is roughly **an order of magnitude slower per tag** and has **no change notification on the wire**, so every subscription is polled.
>
> **Use the S7 module for cyclic process data.** Pick this one for diagnostics and non-tag surfaces, or where network policy permits nothing but HTTPS. The same notice appears at the top of the device configuration form, because that is the last point at which the choice is cheap to change.

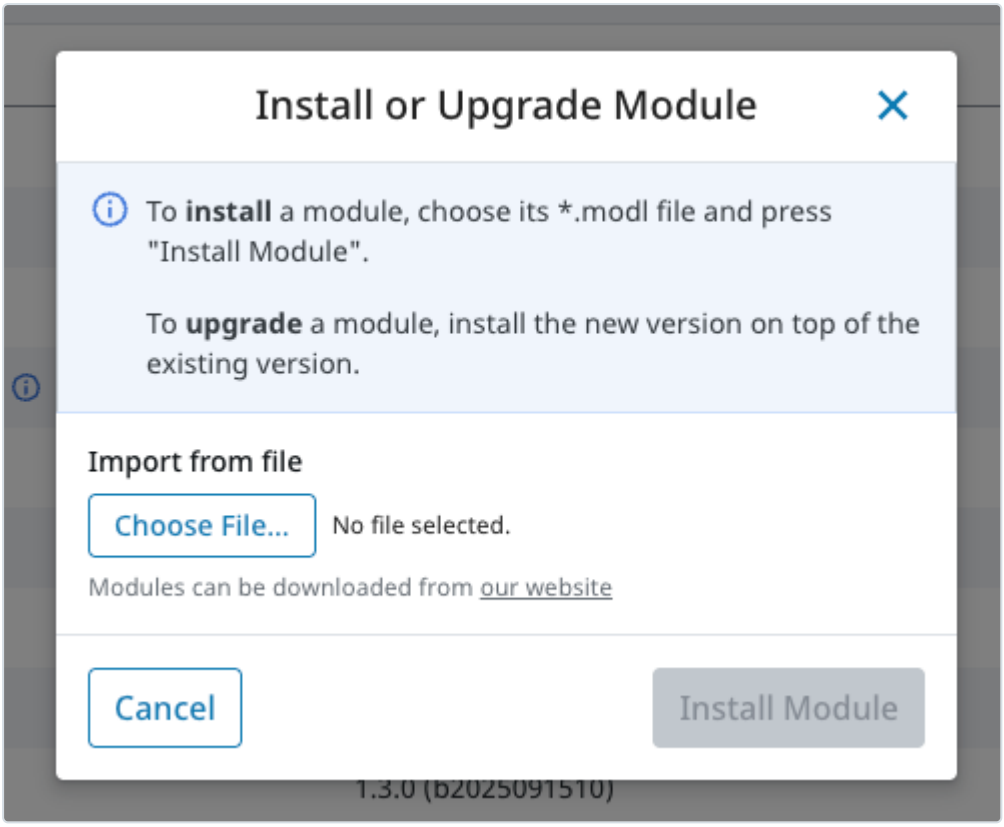
The controller also closes idle connections within seconds. That is expected and handled — the driver establishes connections per operation rather than holding one open — so a device that appears to "reconnect constantly" in the logs is working as designed.



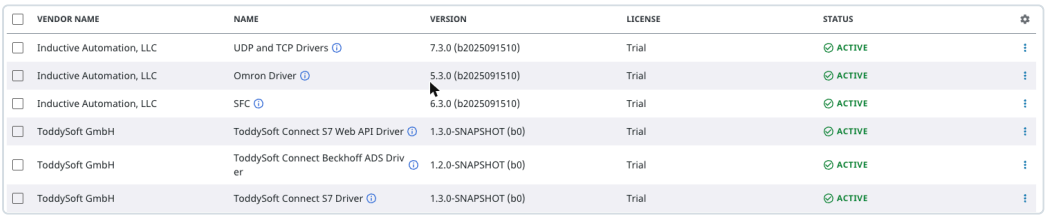
Installation

> **This module is for Ignition 8.3 only.** It is built against the Ignition 8.3 SDK and will not run on any other major version. For Ignition 8.1, install the separate `…-8.1.modl` build instead — the two are not interchangeable, and each gateway needs the build that matches it.
>
> **Ignition 8.0 and earlier are not supported and cannot be.** The module is built on the device-driver API that Ignition introduced with 8.1, so on 8.0 the gateway fails to load it with `NoClassDefFoundError: … opcua.server.api.AbstractDeviceModuleHook`.
>
> Both builds carry the same name in the gateway's module list, so tell them apart by the file you installed or by the module description, which names the Ignition version.
>
> **Ignition's OPC UA module must be installed and running.** This is an OPC UA device driver: it registers with Ignition's own OPC UA server, and every tag it exposes is served through it. With that module absent the gateway cannot load this one, and the log fills with `ClassNotFoundException` / `NoClassDefFoundError` naming `com.inductiveautomation.opcua…` classes. The OPC UA module ships with Ignition and is installed by default, so in practice this bites only on a gateway where it has been removed or disabled. This module declares the dependency as **required** — a mechanism 8.3 has and 8.1 lacks — so Ignition does not attempt to load it at all, and reports the reason instead of failing with a stack trace. In **Status > Diagnostics > Logs** you will see:
>
> `Module "com.toddysoft.connect.…" depends on "com.inductiveautomation.opcua", which is either not installed or inactive on this gateway.`
>
> This is the same handling Ignition's own drivers get in the same situation.

1. Open the Ignition Gateway web interface and navigate to **Config > System > Modules**.
2. Click **Install or Upgrade a Module...**.
3. Select the `ToddySoft-Connect-S7-Web-API-Driver-8.3.modl` file and upload it.



4. Accept the license / certificate prompt to continue the installation.
5. Wait for the Gateway to finish and verify the module appears as **Running** in the modules list.



VENDOR NAME	NAME	VERSION	LICENSE	STATUS
Inductive Automation, LLC	UDP and TCP Drivers	7.3.0 (b2025091510)	Trial	ACTIVE
Inductive Automation, LLC	Omron Driver	5.3.0 (b2025091510)	Trial	ACTIVE
Inductive Automation, LLC	SFC	6.3.0 (b2025091510)	Trial	ACTIVE
ToddySoft GmbH	ToddySoft Connect S7 Web API Driver	1.3.0-SNAPSHOT (b0)	Trial	ACTIVE
ToddySoft GmbH	ToddySoft Connect Beckhoff ADS Driver	1.2.0-SNAPSHOT (b0)	Trial	ACTIVE
ToddySoft GmbH	ToddySoft Connect S7 Driver	1.3.0-SNAPSHOT (b0)	Trial	ACTIVE

Before you start: the controller side

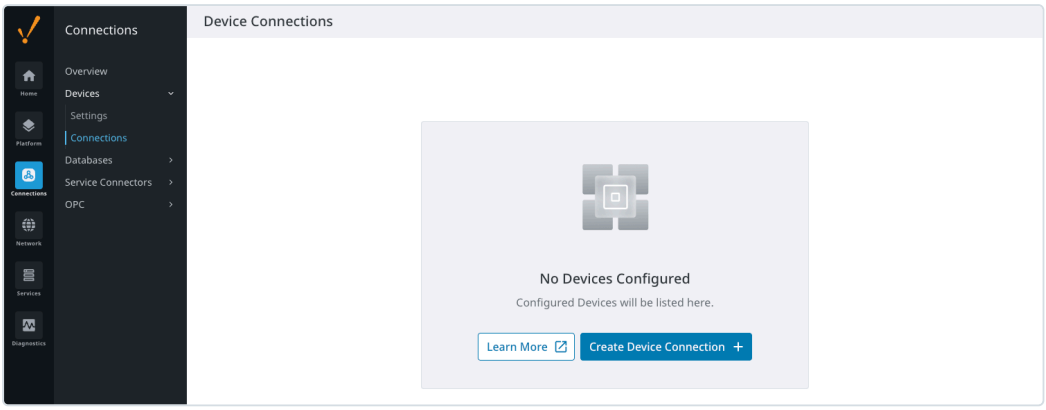
Two things must be true on the controller before a device here can connect, and neither is something this module can do for you:

- **The Web API must be enabled** for the CPU in the TIA project (*Protection & Security* > *Web server*), and the web server itself switched on.
- **The account must hold the Web API permission.** A user that can log into the controller's web pages is not automatically permitted to use the API; the permission is granted separately in the TIA project.

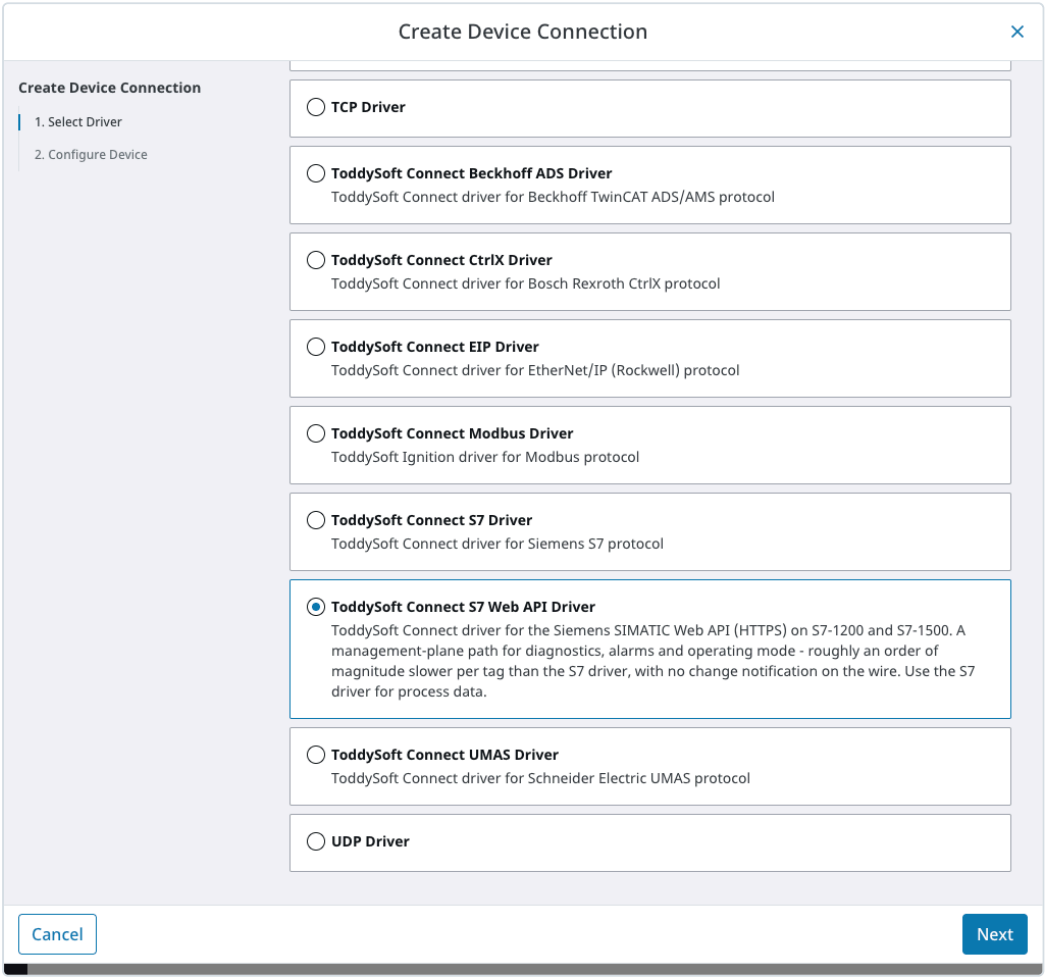
Without the second, the connection succeeds at the TLS layer and the login is then refused — which reads like a wrong password.

Creating a Device Connection

1. Go to **Config > OPC UA > Device Connections**.

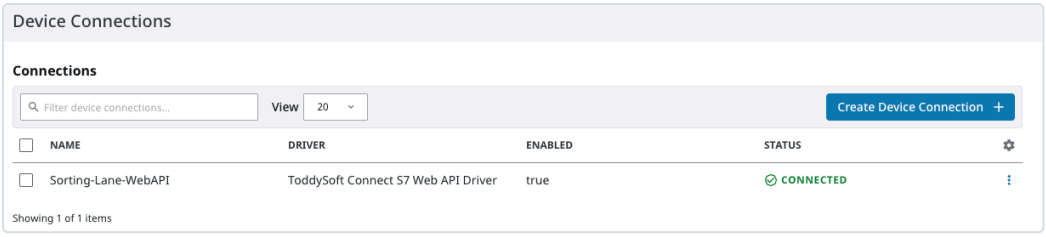


2. Click **Create new Device...** and select **ToddySoft Connect S7 Web API Driver** as the device type.



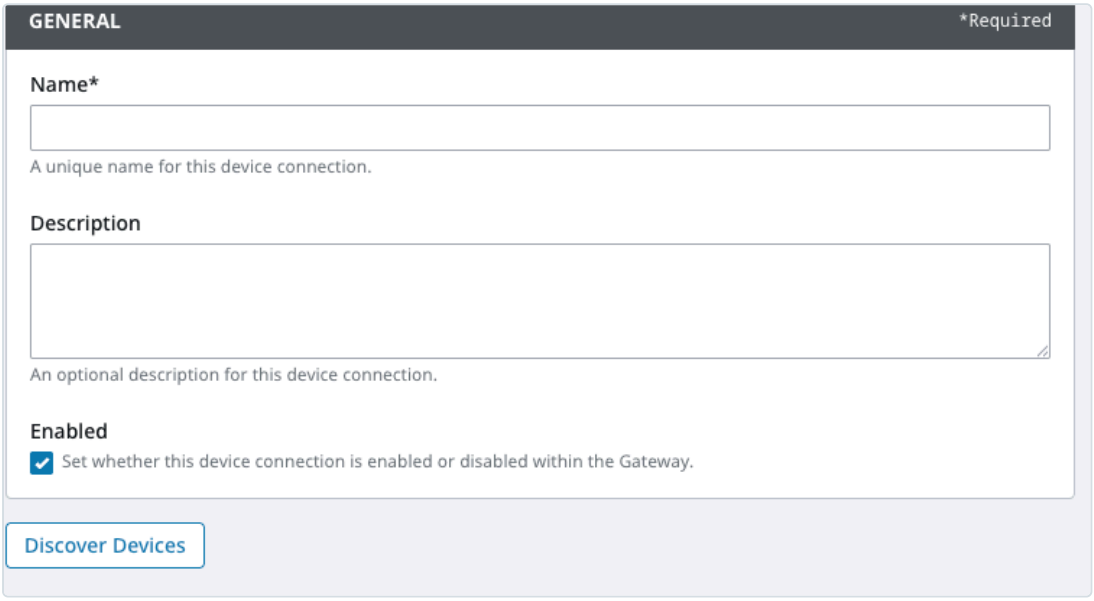
3. Configure the connection settings (see the sections below) and click **Create Device Connection**.

4. The new device then appears in the Device Connections list.



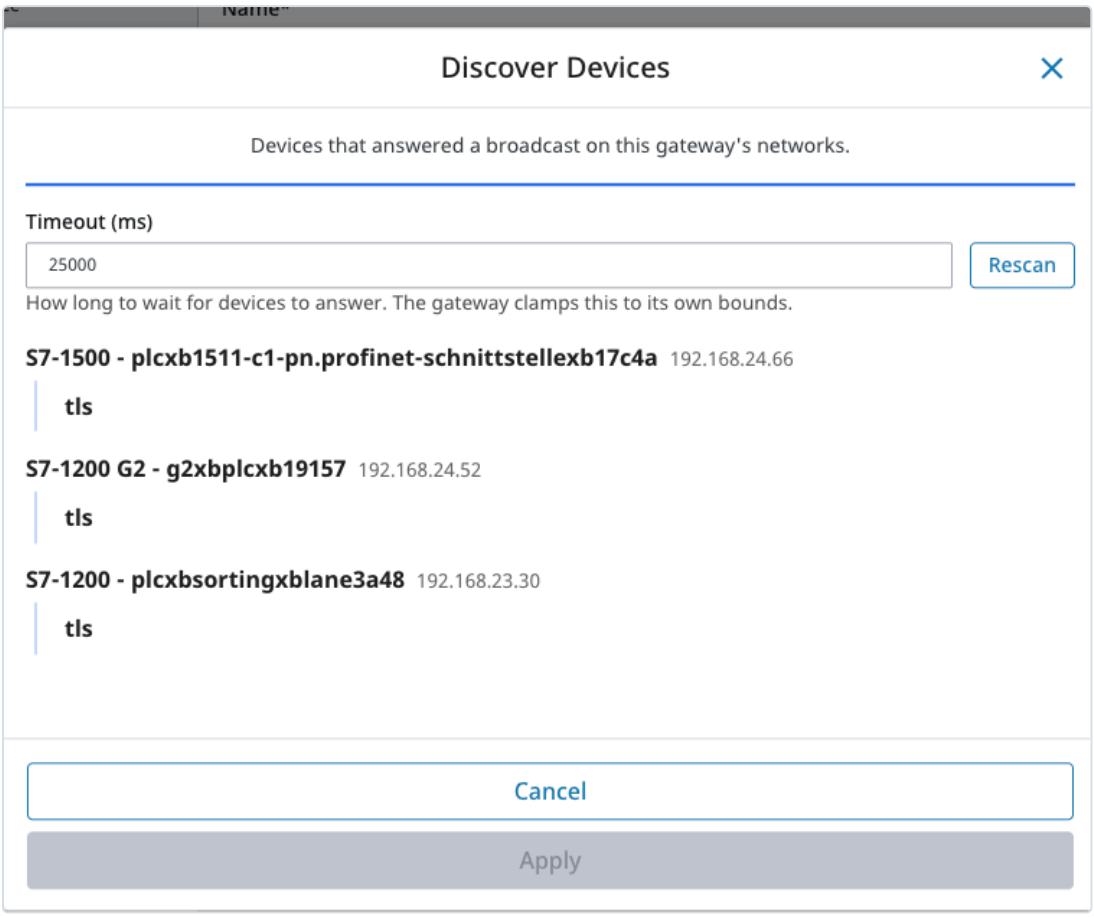
Finding devices on the network

When you create a device, the configuration form carries a **Discover Devices** button directly beneath the **General** section, above the first settings card.



Discover Devices

Sweeps the gateway's own networks for controllers, then asks each one that answers whether it is actually serving the Web API before reporting it. else is not listed, and one that does not answer the probe in time is not listed either — so the result is the set of controllers you can genuinely connect this driver to, rather than everything with an open HTTPS port.



A controller that answers on 443 with something

Selecting a result fills in **Host** and **Port**. Two things it deliberately does not do:

- **It fills in only what the device actually reported.** A field the sweep said nothing about is left alone rather than guessed at.
- **It never fills the username or password.** A broadcast tells you where a controller is, never which account may use its API — and the Web API permission is granted per account in the TIA project, so there is nothing sensible to guess.

Timeout (ms) bounds how long the gateway listens; it defaults to **25000**. The sweep itself takes roughly fifteen seconds, and the per-controller probe allows ten before giving up on a slow TLS handshake. A shorter bound stops the sweep before the answers arrive, and the panel then reports *No devices answered* over a network that is full of controllers.

If applying a result would overwrite something you have already typed, it lists what would change and asks first. Running a sweep needs permission to create device connections; being logged in is not enough.

> **In Docker, discovery needs the discovery agent.** A container sits on Docker's network, not on the plant network the controllers are on, so the search alone finds nothing there. See Running in Docker: the discovery agent; without it, enter the host by hand — everything else on this page works normally.

> **This driver offers no *Test Connection* button.** That action appears only for drivers that publish a device-setup capability the gateway can probe, and this one publishes none. The button is absent rather than present and inert. The Beckhoff ADS module does offer it.

Running in Docker: the discovery agent

A gateway in a Docker container sees only the container's networks, not the plant's: the PROFINET-DCP broadcast behind *Discover Devices* never leaves the container's network. On top of that, Ignition's official Docker image carries no libpcap, which the search needs to send and capture its frames, so in a container it cannot run at all and reports that nothing answered. The **ToddySoft discovery agent** closes that gap. It is a small, separate container that runs on the same machine with the host's network and searches the plant networks on the gateway's behalf. The agent only searches: connecting, browsing and reading stay with the gateway, through the machine's normal routing. A gateway installed directly on the plant machine, or a container on the host network, does not need it.

Installing it. The agent comes as an image archive next to this module, in the share you download the module from: `discovery-agent-<version>-<architecture>.tar.gz`, for `amd64` or `arm64`. Use the agent of the module's version. On the machine that runs the Ignition container (Linux):

```
docker load < discovery-agent-1.3.0-amd64.tar.gz
docker run -d --name toddysoft-discovery-agent \
  --restart unless-stopped --network host --cap-add NET_RAW \
  toddysoft/discovery-agent:1.3.0
```

`--network host` lets it search the plant networks, and `NET_RAW` lets it send and capture the discovery frames. Docker Desktop (macOS, Windows) runs containers in a virtual machine, whose networks the agent would search instead of the plant's.

Nothing to set in Ignition. On every **Discover Devices** the module looks for the agent at its container's default gateway, which on Docker is the machine itself. An agent started later is found on the next search, with no restart. A line above the results says what happened:

- *via discovery agent*: the agent searched; its devices are listed together with any the gateway found itself, each once.
- *no discovery agent found · container networks only*: no agent runs; the search is the gateway's own, as before.
- *discovery agent refused: …* or *discovery agent not usable: …*: the reason follows, for example the licence or the shared secret below.

The panel shows its results after about five seconds, when the agent has finished. The devices listed come from the agent: the gateway's own search cannot run inside the container (see above), which the gateway log notes as *this gateway's own search could not run … the discovery agent answered*. Without an answering agent the panel says why under *Part of the search failed*.

Licence. The agent searches only for a gateway licensed for this driver. With each search the module sends the licence it runs on, the bundled trial included, over an encrypted connection; the agent checks it, uses it for that search and keeps nothing.

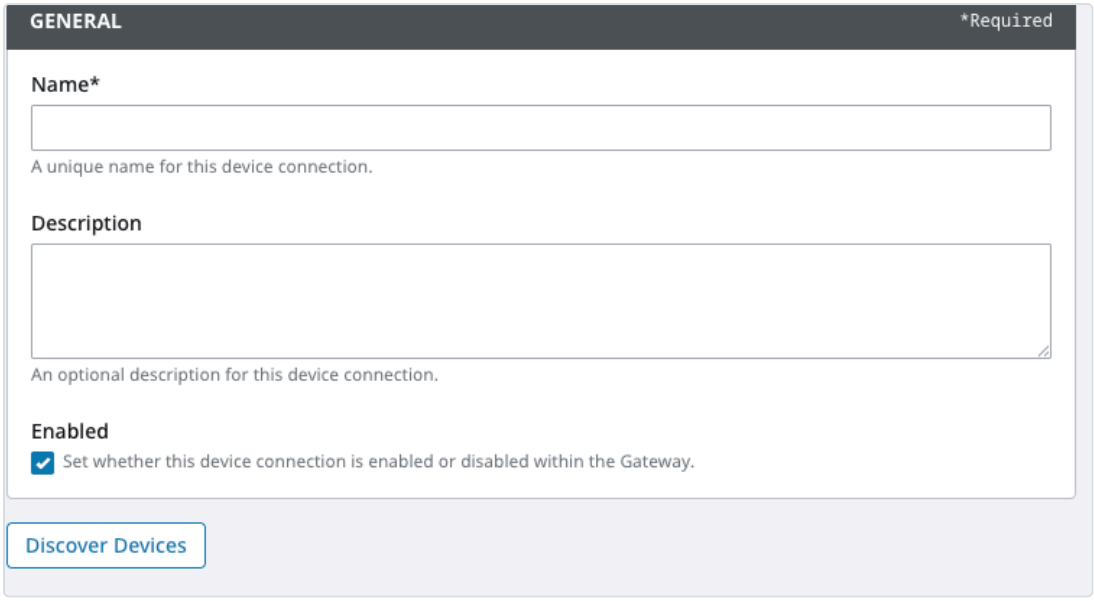
Security. The agent listens only on the Docker networks' own addresses on that machine (port 47900), never on a plant network, and accepts connections only from the container networks. To restrict it further, give it a shared secret: `-e TODDYSOFT_AGENT_SECRET=<secret>` on the agent and `-e TODDYSOFT_DISCOVERY_AGENT_SECRET=<secret>` on the Ignition container. With a secret set, a gateway without it, or with another, is refused. To stop the module looking for an agent, set `TODDYSOFT_DISCOVERY_AGENT=off` on the Ignition container.

Configuration

The configuration form is organized into the following sections, top to bottom: **General**, **Connection**, **Web API Session**, **Certificate Trust**, **Timing**, **Limits** and **Audit Log**.

General

Assigns a name (and optional description) to the connection and controls whether it is enabled.



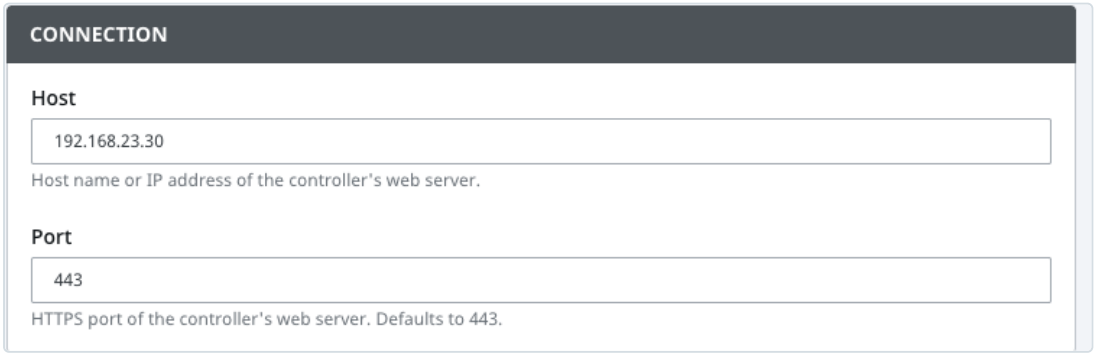
Setting	Description	Default
Name	Unique name for this device connection	
Description	Optional description for this connection	
Enabled	Whether the connection is active	`true`

If `Enabled` is not checked, no connection is established until it is enabled.

Connection

Setting	Description	Default
Host	Host name or IP address of the controller's web server	`localhost`
Port	HTTPS port of the controller's web server	`443`

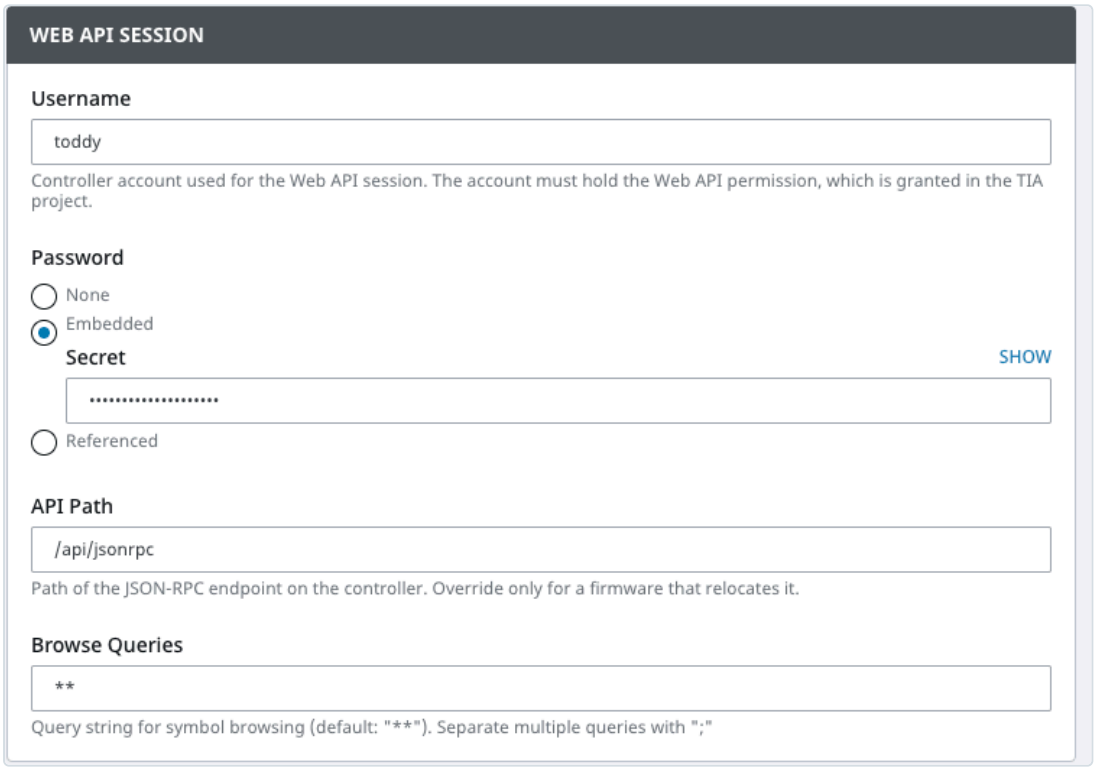
The Web API is HTTPS-only. There is no plaintext mode to fall back to and no transport to select; a plain-HTTP connection is refused. TLS is the driver's only transport, which is why the settings for it appear under *Certificate Trust* rather than behind a transport selector.



Web API Session

Setting	Description	Default
Username	Controller account used for the Web API session	
Password	Password for the Web API account. Redacted from all log output	*None*
API Path	Path of the JSON-RPC endpoint on the controller	`/api/jsonrpc`
Browse Queries	Symbol browse query filter. Separate multiple queries with `;`	`**`

There is no anonymous access — the API authenticates every request, so a username is required. The account must hold the **Web API permission** granted in the TIA project; holding web-server access alone is not enough.



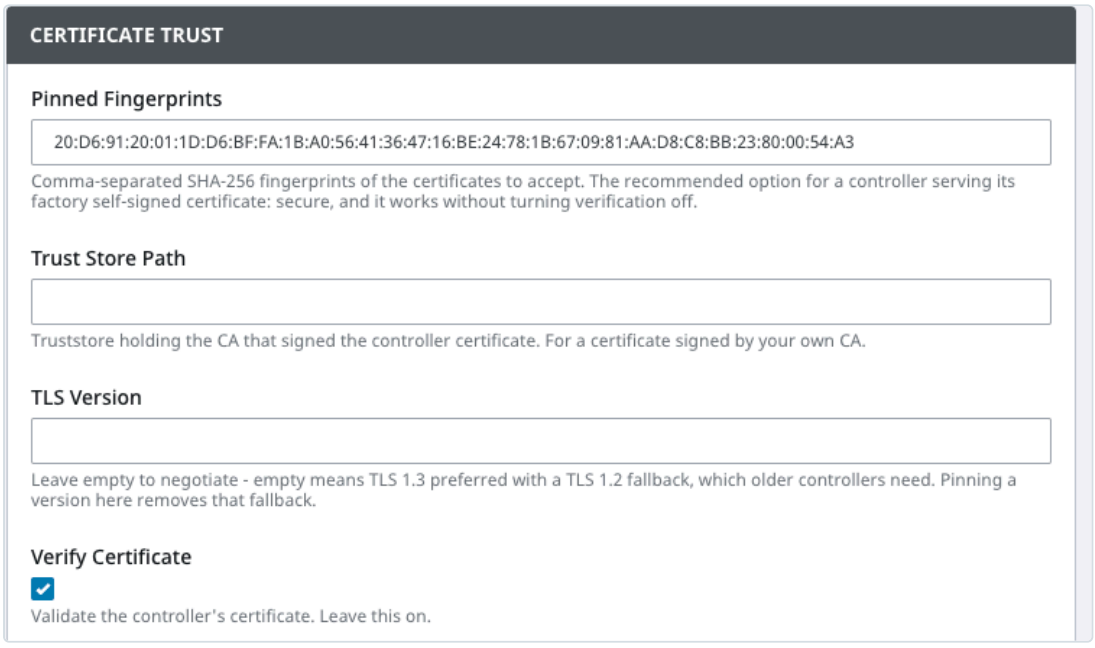
Override **API Path** only for a firmware that relocates the endpoint. The default is correct for every current S7-1200 / S7-1500 firmware.

Since ToddySoft Connect 1.2.0 the credentials are handed to the driver as an object rather than written into the connection string. They no longer reach the gateway log, a support bundle, or any string quoting the connection.

Certificate Trust

Setting	Description	Default
Verify Certificate	Validate the controller's TLS certificate	`true`
Pinned Fingerprints	Comma-separated SHA-256 fingerprints of the certificates to accept	*(empty)*
Trust Store Path	Truststore holding the CA that signed the controller certificate	*(empty)*
Trust Store Password	Password for the truststore. Leave as *None* for an unprotected store	*None*
Trust Store Type	Truststore format: `PKCS12` or `JKS`	`PKCS12`
TLS Version	Pin a protocol version, e.g. `TLSv1.3`. Leave empty to negotiate	*(empty)*

A controller out of the box serves a self-signed certificate, so this decision cannot be avoided. The driver's order of preference is:



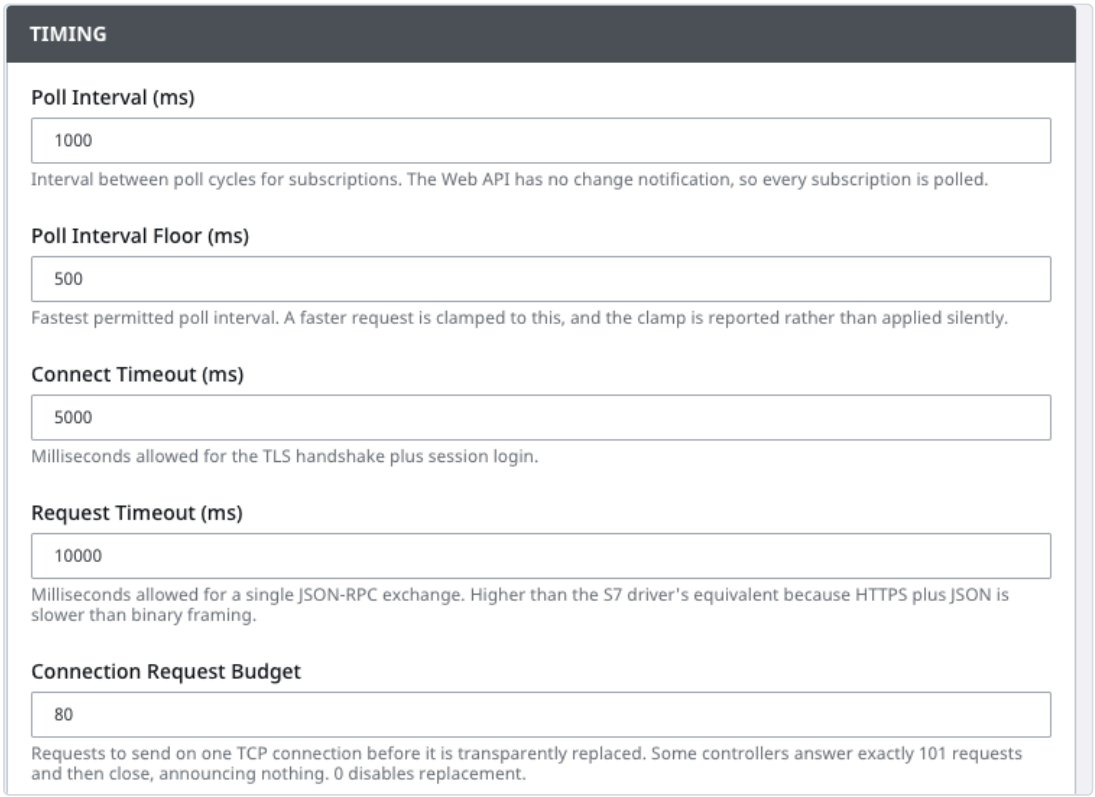
1. **Pinned fingerprints** — the right answer for a factory certificate. It is secure, and it works without turning verification off. When a verification fails, the error message contains the peer's actual fingerprint, ready to paste in.
2. **Your own truststore** — for a certificate signed by your own CA.
3. **The platform trust anchors** — for a certificate from a public CA.
4. **`Verify Certificate` off** — encryption without authentication. Development only.

Leave TLS Version empty unless you have a reason. Empty means TLS 1.3 preferred with a TLS 1.2 fallback, which older controllers need; pinning a version here removes that fallback.

Timing

Setting	Description	Default
Poll Interval (ms)	Interval between poll cycles for subscriptions	`1000`
Poll Interval Floor (ms)	Fastest permitted poll interval	`500`
Connect Timeout (ms)	Milliseconds allowed for the TLS handshake plus session login	`5000`
Request Timeout (ms)	Milliseconds allowed for a single JSON-RPC exchange	`10000`
Connection Request Budget	Requests to send on one TCP connection before it is transparently replaced	`80`

Every subscription is polled. The Web API has no change notification, so *Poll Interval* is the real data rate for this device — not a fallback. Setting it below *Poll Interval Floor* does not fail: the driver clamps the request to the floor and reports the clamp, rather than applying it silently. The form warns you when the interval you have typed is below the floor.



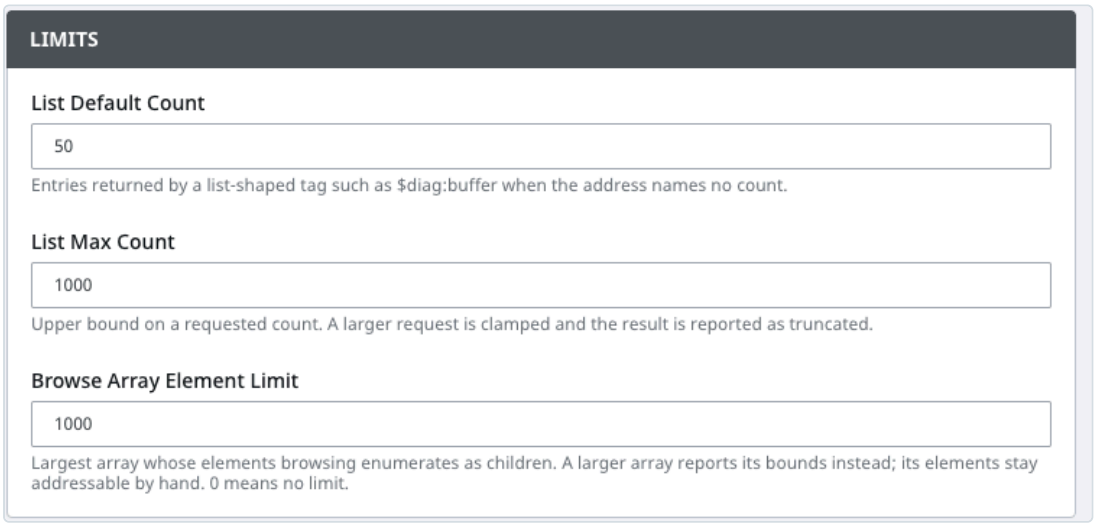
Request Timeout is deliberately higher than the S7 driver's equivalent, because HTTPS plus JSON is slower than binary framing. A JSON-RPC exchange that takes several seconds is not a fault.

Connection Request Budget exists for a real controller behaviour: some firmware answers exactly 101 requests on a connection and then closes it, announcing nothing. Replacing the connection at 80 requests keeps that from ever surfacing as a failed read. Set it to `0` to disable replacement — only worth doing against a controller measured not to need it.

Limits

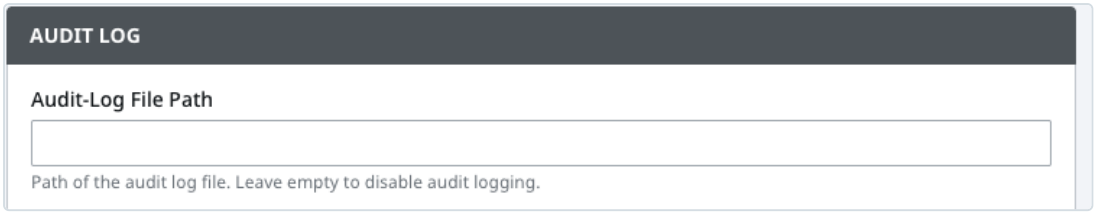
Setting	Description	Default
List Default Count	Entries returned by a list-shaped tag such as `$diag:buffer` when the address names no count	`50`
List Max Count	Upper bound on a requested count. A larger request is clamped and the result is reported as truncated	`1000`
Browse Array Element Limit	Largest array whose elements browsing enumerates as children. `0` means no limit	`1000`

An array larger than **Browse Array Element Limit** reports its bounds instead of children — never a truncated prefix, which would render as an array that is simply smaller than it is. Its elements stay addressable by hand, but the array is **not readable as a whole**: the reader decomposes an aggregate exactly as the browse tree does, so an array the tree will not enumerate is one the reader will not either.



Audit Log

The `Audit Log` writes important diagnostic information to a dedicated log file. This file can grow quickly, so it is advised to only enable it when investigating issues.



Setting	Description	Default
Audit-Log File Path	Absolute path to the audit log file where events will be recorded. Leave empty to disable.	*(empty)*

NOTE: Please ensure the user the `Ignition` process is running under has write access to this path.

Tag Addressing

Symbolic only — there is no absolute-address form. The grammar follows TIA's display form:

```
"<datablock>".<member>[.<member>…][:<TYPE>]
"<plc-tag>"[:<TYPE>]
$<namespace>:<verb>[:<TYPE>][(<count>)]
```

Address	Meaning
`"TestData".g_s16`	An `INT` member of data block `TestData`
`"TestData".g_plant.meta.str`	A nested struct member
`"OPC_UA_DB"."OPC Data".g_b1`	A member whose name is not a plain identifier — **it must be quoted**
`"ManualTrigger"`	A PLC tag in the global tag table
`$diag:buffer[0..19]`	The 20 newest diagnostic-buffer entries

> **Addresses are *not* interchangeable with the S7 module.** That driver accepts `%DB42:0.0:BOOL` absolute forms as well as symbolic ones; this one is symbolic-only and uses the quoted TIA display form. Tags cannot be moved between the two by changing the device.

The type suffix is optional; without one the controller's own type governs, which is the normal case. Members whose names contain anything but letters, digits and `_` — spaces and hyphens are common in real projects — must be quoted, and browsing does this for you.

Browsed addresses are usable verbatim. Browsing also reports array bounds (`ARRAY [1..8] OF BOOL`), string capacity (`STRING(40)`) and write protection, all from the controller's own metadata.

Diagnostics through synthetic tags

Non-tag surfaces are reached through a reserved `$` namespace on the ordinary read/write path — the reason to choose this module in the first place:

Tag	Direction	Meaning
`$diag:buffer` / `$diag:buffer[0..49]`	read	Diagnostic buffer, newest first
`$diag:alarms`	read	Active alarms
`$control:operating-mode:STRING`	read / write	Read the mode, or write `run` / `stop` to request a change
`$session:permissions`	read	The account's effective rights

A list says when it was cut. `$diag:buffer` and `$diag:alarms` return at most **List Max Count** entries, and **List Default Count** when the address names no range. Each list's folder therefore holds three more variables, read with the list and costing no request of their own:

Variable	Type	Meaning
`<list>#list-truncated`	Boolean	`true` when the controller held more entries than came back
`<list>#list-entries-omitted`	Int32	How many were left out; `0` for a complete list
`<list>#list-entries-available`	Int32	The controller's count; empty for a complete list, which does not report it

for example `$diag:buffer#list-truncated`. The **Management** folder reads the buffer as `$diag:buffer`, so it holds **List Default Count** entries and says `true` whenever the controller holds more. A range you address yourself, such as `$diag:buffer[0..19]`, comes back as asked and is cut only above **List Max Count**. Raise **List Default Count** (and **List Max Count** with it) when you need the rest.

A variable literally starting with `$` cannot be addressed — the namespace is reserved.

> **Writing `$control:operating-mode` is plant-affecting, and the acknowledgement does not mean what it looks like.** An `OK` means the request was **queued**, not that the mode changed — and a second request is queued too, not rejected. Measured on an S7-1200 G2: three contradictory commands sent 8–10 ms apart (`run`, `stop`, `run`) were each acknowledged and **all three executed in order**, stopping and restarting the CPU in about 135 ms with no error reported at any point.
>
> - **Never retry a mode change because the acknowledgement was slow.** The first one is already queued; a retry is a second execution.
> - **Guard the UI.** A double-clicked STOP will be carried out twice.
> - **Confirm by reading, not by the acknowledgement.** A read immediately after the write reports the *old* mode, which is indistinguishable from a refusal. Poll until it settles and treat a timeout as the failure.

Arrays and structures

Aggregates read whole; they never write whole. The wire has no aggregate transfer: reading an array by name is refused by the controller, and so is reading a structure. The driver serves reads anyway, by decomposing the aggregate into the addresses the controller *does* accept and reassembling the answer. Writes stay refused, deliberately — a decomposed read that fails partway is a retryable bad observation, while a decomposed write that fails partway leaves the controller holding a mixture of new and old members with no rollback. **Write the members or the elements individually.**

One failed member fails the whole tag, with the message naming the member; a structure delivered with holes would look complete while quietly carrying nulls.

Three details matter if you compose subscripts by hand rather than browsing:

- **Indices are comma-separated inside one bracket group.** `g_matI16_2x3[1,1]` works; `g_matI16_2x3[1][1]` is rejected as an invalid address.
- **The declared lower bound is preserved, never normalised.** In the same block one array can start at 0 and another at 1. Assuming a base addresses the wrong element.
- **An intermediate row of a multi-dimensional array is itself an array** — readable by decomposition, not writable.

Configure **Browse Queries** to control which symbols appear when browsing tags in the Ignition Designer.

Tag Groups

By default, all tags are collected in the default tag group, which means all selected tags are fetched at the default interval of once a second.

If different intervals are desired, create additional tag groups, configure the desired intervals there, and assign tags to the corresponding tag groups.

On this driver the tag group interval interacts with *Poll Interval*. The Web API is polled, so a tag group faster than the device can serve does not produce faster data — it produces requests that queue. Size the tag groups to what the controller can answer, not to what you would like.

Licence diagnosis

When a device reports a licence problem, the module explains it on the gateway itself. Open the device's settings (**Config > OPC UA > Device Connections**, edit the device, or create a new one): the **Licence** card at the top of the form shows the diagnosis. It is on the form for a new device too, so the licence can be checked before the first device exists.

The diagnosis shows:

- **which gateway** this is, and its role if it is part of a redundant pair;
- **Ignition's licence** for this module on this gateway: activated, trial running (with its end) or trial expired;
- **what ToddySoft licence data arrived** with Ignition's licence: how many parts, whether they are complete, and a checksum that identifies the licence without revealing it;
- the **driver's own verdict** and the **cause**, in words.

Cause	Meaning	What to do
`IGNITION_UNLICENSED`	Ignition applies no activated licence to this module on this gateway (trial running or expired). ToddySoft licence data only arrives with an activated licence.	Activate a licence key that covers this module on this gateway. In a redundant pair every node needs its own activation. If a key was activated and this cause remains, see Troubleshooting: an older, expired key covering this module may be the one Ignition judges it by.

Cause	Meaning	What to do
`NO_DATA_FOR_DRIVER`	The activated key carries no ToddySoft licence for this driver.	Send the report to ToddySoft.
`DATA_INCOMPLETE`	The licence data arrived incomplete or damaged.	Send the report to ToddySoft; the licence has to be delivered again.
`DATA_REJECTED`	The driver refused the delivered licence; the reason is shown.	Send the report to ToddySoft.
`RUNNING_ON_BUNDLED_TRIAL`	A licence was delivered but not accepted, and the driver runs on its built-in trial until that ends.	Send the report to ToddySoft.
`NOT_CHECKED`	Licence data arrived, but the driver gave no verdict - for instance because it could not be loaded.	Send the report to ToddySoft.
`VALID`	Licensed; tier and expiry are shown.	Nothing.
`NOT_YET_KNOWN`	The first seconds after start-up, before Ignition reports a licence state.	Wait a moment.

Sending a report to support. The diagnosis comes as a plain-text report, the one thing to send with any licence question. **Copy report** puts it on the clipboard; **Show report** displays it. The same text is served at `http://<gateway>:<port>/data/com.toddysoft.connect.s7webapi/licence-report` to a logged-in user. It contains no licence data, no keys, no credentials and no customer details. In a redundant pair, send the report from both nodes.

The gateway log carries the same report once each time the licence situation changes.

Troubleshooting

Licensing problems after a new or renewed licence key was activated — `DATA_INCOMPLETE` with "duplicate fragment index", or `IGNITION_UNLICENSED` with "trial expired", in the licence diagnosis:

- **Two licence keys covering this module are activated on the gateway at the same time**, typically a renewed or upgraded key activated alongside the old one. Ignition hands the module the licence data of both, in numbered parts that carry the same numbers; the module cannot tell which parts belong together, keeps one of each pair, and the licence it reassembles is unreadable. Depending on the parts it keeps, devices are refused or run on the module's bundled trial until that ends.
- If one of the two keys has **expired**, Ignition may judge the module by that key alone: the diagnosis shows `IGNITION_UNLICENSED` ("trial expired - no activated licence applies to this module") and "Data: none received", although the new key was activated successfully.
- Keep exactly **one** activated licence key that covers this module per gateway: deactivate the old one under **Platform > Licensing**, then restart the module (or the gateway). In a redundant pair, check every node.

The module will not load, or the log shows `ClassNotFoundException` / `NoClassDefFoundError`:

- Ignition's **OPC UA** module is missing or disabled. This driver is an OPC UA device driver and cannot load without it. Install or re-enable it under **Config > System > Modules** and restart the gateway — see Installation

Device shows "Faulted":

- Verify the controller's host name / IP is reachable on the configured HTTPS port
- Check that the **Web API is enabled** for the CPU in the TIA project
- Review Gateway logs: **Status > Diagnostics > Logs**

Login fails although the password is right:

- The account most likely lacks the **Web API permission**, which is granted separately from web-server access in the TIA project. The TLS layer succeeds and the login is refused, which reads exactly like a wrong password.

TLS connection fails:

- A factory self-signed certificate cannot pass ordinary verification. Pin its fingerprint under **Pinned Fingerprints**; the error message reports the controller's actual fingerprint
- For a certificate signed by your own CA, set **Trust Store Path** and **Trust Store Type**
- For an older controller, clear **TLS Version** so the TLS 1.2 fallback is available again

The logs show the connection being opened and closed repeatedly:

- Expected. The controller closes idle connections within seconds, so the driver establishes them per operation rather than holding one open. This is not a fault and needs no configuration change.

Reads fail after roughly a hundred requests:

- Some firmware answers exactly 101 requests on a connection and then closes it, announcing nothing. **Connection Request Budget** (default `80`) replaces the connection before that happens; if it has been set to `0`, restore the default.

Reading an array or a structure returns an error:

- Whole-aggregate *writes* are unsupported by design — write the members or elements individually. If a *read* fails, one member failed and the message names it. An array above **Browse Array Element Limit** is not readable whole at all; its elements remain individually readable.

Collection is too slow:

- This driver is roughly an order of magnitude slower per tag than the S7 driver and has no change notification. If the cycle time matters, that is the wrong module for the job — use the S7 module for process data and keep this one for diagnostics.

Docker related issues

Ordinary connections work normally from a container: the Web API is an outbound HTTPS connection with no identity for this end to announce. Only **Discover Devices** is affected: its broadcast never leaves the Docker network. See Running in Docker: the discovery agent.

VPN related issues

A remote controller reached over a VPN needs nothing special from this driver. If the connection fails, it is ordinary reachability — confirm that the HTTPS port is routed and not blocked.

Version history

Unreleased — 1.3.0

Built on ToddySoft Connect 1.3.0.

- **Added:** this manual.
- **New:** licence diagnosis — why a device is or is not licensed, shown on the gateway, with a report to send to support. See Licence diagnosis.

- **New:** troubleshooting for two licence keys covering this module activated on one gateway at the same time. See Troubleshooting.
- **New:** discovery when Ignition runs in Docker, through the ToddySoft discovery agent. See Running in Docker: the discovery agent.
- **New:** a diagnostic list (`$diag:buffer` , `$diag:alarms`) says when it was cut short: three variables in its folder, `#list-truncated` , `#list-entries-omitted` and `#list-entries-available` . See Diagnostics through synthetic tags.
- **Changed:** date and time tags have new value types. DATE and DATE_AND_TIME are DateTime values (until now a DateTime tag holding epoch milliseconds as a number); TIME and LTIME are milliseconds, TIME_OF_DAY and LTIME_OF_DAY milliseconds since midnight (Int64). Writes take the same forms. Adjust scripts and bindings that used the old values.
- **Fixed:** a write the PLC refuses now fails in Ignition, with the reason as the status (for example `Bad_TypeMismatch` for a value of the wrong type). Until now every write was reported as successful, and the refusal appeared only in the gateway log.
- **Fixed:** a tag that has not been read yet, or whose last read failed, no longer reads 0 with Good quality. Before its first read it has no value and the quality Uncertain; after a failed read it keeps its last value with Bad quality until the next good read.
- **Fixed:** removing the last tag of a tag group (or polling rate) no longer leaves the device showing *Connection Lost*. A poll still running at that moment made the device fail with `Batch '...' not found` in the gateway log and stay in that state until it was restarted.
- **Changed:** a browsed array is one folder, named after the array and its index range (for example `arr[0..9]`), holding its elements; until now the elements sat directly beside the array's neighbours. A multi-dimensional array, until now left out of the browse, appears as nested folders down to its elements (`cube[1..2][1..2]` , then `cube[1][1..2]` , then `cube[1][1]`). An array of structures shows a folder per element holding its members (for example `pts[2].x`). Each element keeps its OPC item path, so existing tags keep working.